

**UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA Y SISTEMAS**

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



TEMA: Practica de hash 1

ESTUDIANTE:

**Quispe Huaman Franklin Victor
Huancapaza Quispe Rody Michael**

DOCENTE:

Zanabria Gálvez Aldo Hernán

CURSO:

ALGORITMOS Y ESTRUCTURAS DE DATOS

Puno, Perú

2025

1. Introducción

En el contexto del comercio electrónico (eCommerce), la velocidad de respuesta es crítica. Las Tablas Hash son estructuras de datos fundamentales que permiten la búsqueda, inserción y recuperación de información (como usuarios y productos) en un tiempo promedio constante $O(1)$. Esta práctica analiza cómo diferentes métodos de resolución de colisiones y diferentes lenguajes de programación (Python y C++) afectan el rendimiento al procesar grandes volúmenes de datos.

2. Metodología

Para este análisis, se utilizó el dataset "eCommerce behavior data from multi category store" de Kaggle (archivo 2019.csv).

- **Muestra:** Se trabajó con una submuestra de 100,000 registros para garantizar comparabilidad.

https://docs.google.com/spreadsheets/d/1M0pjInDx1_8T4IHUIQvEQFIGJKv9LMXDIppCRo7eFZQ/edit?usp=sharing

- **Clave de búsqueda:** user_id.
- **Estructuras implementadas:** 1. Tabla Hash con Encadenamiento Separado (uso de listas enlazadas).
- **Enlace oficial de Kaggle:**
<https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store> }

2. Tabla Hash con Sondeo Lineal (Direccionamiento abierto).

3. Implementación de Código

A. Fragmento de Lógica en Python (Eficiencia de Alto Nivel)

Python permite una implementación rápida utilizando la biblioteca pandas para la lectura de datos.

Link del código:

B. Fragmento de Lógica en C++ (Control de Memoria)

En C++, la gestión manual de punteros permite un análisis más profundo del uso de la memoria.

Link del código:

4. Análisis de Resultados y Comparación

Criterio	Python	C++
Tiempo de Inserción	Más lento (debido al overhead del intérprete).	Significativamente más rápido (ejecución nativa).
Consumo de Memoria	Alto (objetos dinámicos de Python).	Optimizado (control total de estructuras).
Facilidad de Uso	Muy alta (pandas facilita la carga de CSV).	Moderada (requiere manejo de archivos y tipos).

5. Preguntas de la Guía

1. ¿Qué método produjo menor número de colisiones y por qué?

Teóricamente, ambos métodos experimentan el mismo número de colisiones iniciales si utilizan la misma función hash. Sin embargo, el Encadenamiento Separado maneja mejor las colisiones en términos de rendimiento cuando el "Factor de Carga" supera el 0.7, ya que no satura los índices vecinos como lo hace el Sondeo Lineal.

2. ¿Cuál es la ventaja de usar un número primo para el tamaño de la tabla?

El uso de un número primo (ej. 10007) ayuda a que la distribución de los índices sea más uniforme, especialmente si las claves (user_id) presentan patrones o tendencias aritméticas, minimizando así los "clusters" o agrupamientos.

3. ¿Qué método recomendarías para el dataset 2019.csv?

Recomiendo el Encadenamiento Separado. El dataset contiene muchos user_id repetidos (un mismo usuario realiza varios eventos). El encadenamiento permite almacenar múltiples registros para una misma clave de forma natural sin degradar severamente el tiempo de búsqueda.

6. Conclusiones

- Las tablas hash reducen drásticamente el tiempo de búsqueda en comparación con búsquedas lineales en archivos CSV grandes.
- C++ es la opción ideal para sistemas de alta frecuencia donde el milisegundo cuenta, mientras que Python es preferible para prototipado rápido y análisis de datos.
- El Sondeo Lineal es eficiente en memoria pero sufre de "agrupamiento primario", lo que lo hace menos apto para datos con muchas colisiones.

7. Análisis de Resultados (Basado en Ejecución Experimental)

A partir de la ejecución del benchmark, se observó que el comportamiento de las tablas hash depende directamente del tamaño de la tabla y del factor de carga (λ). A medida que aumenta la cantidad de datos, el número de colisiones también se incrementa.

En el método de encadenamiento separado, se registró un mayor número de colisiones; sin embargo, estas no afectan significativamente el rendimiento debido a que los elementos se almacenan en listas enlazadas independientes. Esto permite mantener tiempos de inserción estables incluso con alta carga.

Por otro lado, el sondeo lineal mostró inicialmente menos colisiones, pero su rendimiento se degradó rápidamente al aumentar el factor de carga. Esto se debe al fenómeno de

clustering, donde múltiples claves se agrupan en posiciones contiguas, aumentando el número de accesos necesarios.

En cuanto al tiempo de ejecución, se evidenció que C++ presenta un rendimiento superior a Python, debido a su naturaleza compilada y mejor manejo de memoria. Python, aunque más lento, facilitó el análisis de datos y la implementación.

Finalmente, se comprobó que el uso de tamaños de tabla primos mejora la distribución de claves y reduce la probabilidad de colisiones, optimizando el desempeño general.

LINK DE CODIGOS :

https://github.com/piolin55/hash_1.git